UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE LA COSTA

ACTIVIDAD DE INVESTIGACIÓN

Pruebas No Funcionales, Registro y Seguimiento de Defectos, y Pruebas de Regresión

Materia: 2026 B2 Laboratorio Abierto, Pruebas
Profesor: José Francisco Reinaga Camacho
Fecha de entrega: 14 de octubre de 2026

Integrantes del Equipo:
222967908 — Corona Méndez Jonathan Ismael
222967843 — González Crisanto Ana Karen
222967835 — Herrera Galindo Brithany Juliette

Puerto Vallarta, Jalisco — Octubre de 2026

Objetivo de la Actividad

Comprender a profundidad la importancia crítica de las pruebas no funcionales, los mecanismos formales de registro y seguimiento de defectos, y las estrategias de ejecución de pruebas de regresión dentro del proceso de aseguramiento de la calidad del software (Software Quality Assurance - SQA). A través de esta investigación se analiza cómo estos tres pilares se complementan para garantizar sistemas informáticos robustos, seguros, escalables y con alta mantenibilidad en entornos de producción reales [1], [2].

1. Investigación Teórica

1.1. Pruebas No Funcionales

Concepto y propósito: Las pruebas no funcionales corresponden a aquellas actividades de verificación y validación orientadas a evaluar los atributos de calidad, comportamiento y restricciones operativas de un sistema de software, en contraposición con las funciones o cálculos específicos que este realiza [3]. Mientras que las pruebas funcionales responden a la interrogante "¿Qué hace el sistema?", las pruebas no funcionales responden a "¿Cómo lo hace?" (es decir, con qué rapidez, bajo qué nivel de seguridad, ante cuánta carga simultánea y con qué grado de facilidad de uso) [4].

El propósito primordial de las pruebas no funcionales radica en garantizar que la aplicación sea viable, confiable y satisfactoria para los usuarios finales dentro de un entorno de producción real. Un producto de software puede ejecutar con precisión matemática cada regla de negocio, pero si tarda minutos en responder, presenta vulnerabilidades que comprometen datos confidenciales o colapsa al recibir múltiples conexiones concurrentes, dicho producto resulta completamente inútil y perjudicial para la organización [3], [5].

Tipos Principales de Pruebas No Funcionales

De acuerdo con modelos internacionales de calidad como el estándar ISO/IEC 25010 (Systems and software Quality Requirements and Evaluation - SQuaRE) [3], las pruebas no funcionales se clasifican en diversas categorías esenciales:

- **a) Pruebas de Rendimiento (Performance Testing):** Evalúan el tiempo de respuesta, la latencia, el rendimiento transaccional (throughput) y el consumo de recursos (CPU, memoria RAM, I/O en disco, ancho de banda de red) bajo condiciones definidas de trabajo. Se subdividen a su vez en: (1) Pruebas de Carga (Load Testing), para medir el comportamiento con la concurrencia normal esperada; (2) Pruebas de Estrés (Stress Testing), que someten al sistema a demandas extremas más allá de su capacidad nominal para identificar su punto de quiebre y su capacidad de recuperación; (3) Pruebas de Resistencia o Fatiga (Endurance/Soak Testing), que monitorean la estabilidad durante periodos prolongados para detectar fugas de memoria (memory leaks); y (4) Pruebas de Picos (Spike Testing), que analizan la respuesta frente a incrementos repentinos de tráfico.

- **b) Pruebas de Seguridad (Security Testing):** Verifican que la aplicación y sus canales de comunicación protejan adecuadamente la confidencialidad, integridad, disponibilidad, autenticidad y no repudio de la información. Incluyen análisis de vulnerabilidades (basados en estándares como OWASP Top 10), pruebas de penetración (pentesting), auditorías de autenticación multifactor, control de acceso basado en roles (RBAC), mitigación de inyecciones SQL/NoSQL, blindaje contra Cross-Site Scripting (XSS), falsificación de peticiones en sitios cruzados (CSRF) y cifrado de datos tanto en reposo como en tránsito mediante TLS/HTTPS [1].
- **c) Pruebas de Usabilidad (Usability Testing):** Miden el grado de facilidad de aprendizaje (learnability), eficiencia operativa, memorabilidad, tolerancia a fallos por parte del usuario y satisfacción subjetiva al interactuar con la interfaz humana. Se fundamentan en principios ergonómicos, heurísticas de usabilidad de Nielsen y estándares de accesibilidad web (como WCAG 2.1) para asegurar que usuarios con diversas capacidades puedan operar el sistema sin fricción.
- **d) Pruebas de Compatibilidad e Interoperabilidad:** Aseguran que el software opere de manera consistente y uniforme a través de diferentes plataformas, sistemas operativos (Windows, Linux, macOS, Android, iOS), navegadores web (Chrome, Firefox, Safari, Edge), resoluciones de pantalla y entornos de hardware. Asimismo, validan el intercambio seguro y transparente de datos con servicios externos, APIs y bases de datos heterogéneas.
- **e) Pruebas de Confiabilidad y Recuperabilidad (Reliability & Disaster Recovery):** Comprueban la capacidad del sistema para mantener un nivel especificado de servicio y disponibilidad ante condiciones anómalas o caídas de infraestructura. Evalúan métricas como el tiempo medio entre fallos (MTBF) y el tiempo medio de recuperación (MTTR), así como la efectividad de los respaldos automatizados y los mecanismos de conmutación por error (failover).

Diferencias entre Pruebas Funcionales y No Funcionales

Aunque ambas modalidades son indispensables para certificar la calidad integral de una solución tecnológica, poseen enfoques, objetivos, orígenes de requisitos y métodos de evaluación sustancialmente distintos [4], [5]. A continuación, se presenta una tabla comparativa que sintetiza sus contrastes fundamentales:

Criterio de Evaluación	Pruebas Funcionales	Pruebas No Funcionales
Pregunta Fundamental	¿Qué hace el sistema? (Verifica el cumplimiento de requerimientos funcionales y reglas de negocio).	¿Cómo lo hace el sistema? (Evalúa atributos de calidad como velocidad, seguridad y usabilidad).
Objetivo Primordial	Validar entradas y salidas del sistema frente a las especificaciones esperadas.	Asegurar la estabilidad, experiencia de usuario y resiliencia en condiciones reales.
Base de Especificación	Casos de uso, historias de usuario,	Atributos de calidad (ISO/IEC 25010),

	requerimientos funcionales del cliente.	SLAs (Acuerdos de Nivel de Servicio) y normativas de seguridad.
Técnicas Comunes	Partición de equivalencia, análisis de valores límite, tablas de decisión, pruebas unitarias.	Pruebas de carga con usuarios virtuales, escaneo de vulnerabilidades, pruebas de estrés térmico/memoria.
Métricas Típicas	Tasa de casos aprobados/reprobados, porcentaje de cobertura de código, defectos funcionales.	Tiempo de respuesta (ms), transacciones por segundo (TPS), tasa de errores HTTP, uso de CPU y RAM.
Ejecución Manual	Frecuente y viable tanto de forma manual como automatizada mediante scripts.	Compleja de simular manualmente; requiere herramientas especializadas (ej. JMeter, k6, ZAP).
Momento Típico de Inicio	Desde las etapas tempranas de codificación (pruebas unitarias y de módulo).	Frecuentemente tras contar con módulos integrados, aunque la seguridad y rendimiento se perfilan desde la arquitectura.

1.2. Registro y Seguimiento de Defectos (Defect Tracking)

¿Qué es un defecto en software? De acuerdo con la terminología estandarizada del glosario IEEE Std 829 y el estándar ISO/IEC 24765 [1], [6], es crucial distinguir tres conceptos íntimamente relacionados pero conceptualmente distintos:

- **Error (Error / Mistake):** Es una acción humana equivocada cometida por un analista, diseñador o programador (por ejemplo, una mala interpretación de una regla de cálculo fiscal o un error tipográfico en la lógica condicional).
- **Defecto o Bug (Fault / Defect):** Es la manifestación estática del error en el artefacto de software (código fuente, scripts de base de datos o documentación de diseño). Constituye una imperfección interna que tiene el potencial de provocar una falla cuando se ejecuta la ruta afectada.
- **Fallo o Falla (Failure):** Es la discrepancia dinámica visible entre el comportamiento observado del sistema durante su ejecución y el comportamiento especificado en los requerimientos. Ocurre cuando el flujo de control del programa alcanza y activa el defecto en tiempo de ejecución.

Etapas del ciclo de vida de un defecto (Bug Life Cycle): El ciclo de vida de un defecto describe la secuencia formal de estados y transiciones por los que transita una anomalía desde que es detectada por primera vez hasta que se verifica su solución y se archiva de manera concluyente [5], [6]. Los estados típicos en los flujos de ingeniería de software son:

- **1. Nuevo (New):** El tester o usuario descubre una discrepancia, reúne las evidencias (capturas, logs, pasos de reproducción) y la registra en la herramienta.
- **2. Asignado (Assigned):** El líder de QA o el gestor de proyecto revisa el reporte, valida su legitimidad, asigna severidad/prioridad y lo turna al desarrollador correspondiente.

- **3. Abierto / En Progreso (Open / In Progress):** El desarrollador asignado investiga el defecto, reproduce el comportamiento en su entorno local y analiza la causa raíz del problema.
- **4. Resuelto / Corregido (Fixed / Resolved):** El programador implementa la corrección en el código, añade o actualiza pruebas unitarias para prevenir regresiones y despliega el cambio al entorno de pruebas.
- **5. En Espera de Retest (Pending Retest):** La corrección se encuentra lista y disponible en el ambiente de aseguramiento de calidad para que el equipo de pruebas verifique el parche.
- **6. Re-evaluación (Retest / Testing):** El tester ejecuta nuevamente los pasos de reproducción sobre la versión corregida para comprobar si la anomalía desapareció.
- **7. Cerrado (Closed):** Al comprobar que la falla ha sido completamente subsanada y no introduce efectos adversos, el estado del defecto pasa a cerrado y documentado.
- **8. Reabierto (Reopened):** Si durante la re-evaluación el defecto persiste o la solución fue parcial, el ticket regresa al estado de abierto con nuevas anotaciones técnicas.
- **Estados Alternos:** Rechazado (Rejected / Not a Bug, cuando el sistema funciona según lo diseñado), Duplicado (Duplicate, si ya existía un reporte previo) o Diferido (Deferred, cuando se pospone su resolución a un sprint futuro por baja prioridad).

Herramientas comunes para gestionar defectos: La gestión moderna de incidencias requiere plataformas que centralicen la trazabilidad, asignación de responsabilidades, categorización por severidad y comunicación interdisciplinaria entre desarrollo, operaciones y control de calidad [5]. Entre las más representativas destacan:

- **Jira (Atlassian):** El estándar de facto en entornos corporativos ágiles (Scrum y Kanban). Permite la configuración detallada de flujos de trabajo personalizados, campos obligatorios para reproducción, niveles de severidad/prioridad, estimación de esfuerzos e integración directa con repositorios Git (GitHub, GitLab, Bitbucket) y pipelines de CI/CD.
- **Bugzilla (Mozilla):** Una de las herramientas de código abierto más veteranas, robustas y confiables para el seguimiento riguroso de errores. Destaca por su potente motor de búsqueda avanzada, auditoría completa de cambios mediante correo electrónico y soporte estricto para jerarquías de dependencias entre bugs.
- **TestLink:** Sistema web de gestión de pruebas (Test Management System) enfocado primordialmente en la administración de planes de prueba, suites y casos de prueba. Aunque su núcleo es el diseño de pruebas, se integra de forma bidireccional con rastreadores de incidencias como Jira, Bugzilla y Redmine para trazar casos de prueba con defectos.
- **GitHub Issues / GitLab Issues:** Soluciones nativas de gestión integradas en los propios repositorios de código. Facilitan vincular un defecto directamente con ramas de trabajo, Pull Requests y commits específicos (ej. "fixes #142"), posibilitando el cierre automático del ticket una vez que el código corregido es integrado a la rama principal.

1.3. Pruebas de Regresión (Regression Testing)

Objetivo y cuándo se aplican: El propósito primordial de las pruebas de regresión consiste en verificar que las modificaciones recientes introducidas en el software —tales como la adición de nuevas características, la optimización de código, parches de seguridad, refactorizaciones o la corrección de un bug— no hayan provocado efectos colaterales indeseados ni hayan corrompido funcionalidades existentes que previamente operaban de manera correcta [1], [4].

Las pruebas de regresión se deben aplicar de manera disciplinada en situaciones clave del ciclo de vida del software:

- **a) Tras la corrección de un defecto:** Para asegurar que la modificación aplicada no quebrantó otra rutina vinculada directa o indirectamente.
- **b) Al incorporar nuevas funcionalidades:** Para garantizar que los nuevos módulos o endpoints no alteren los flujos comerciales o de autenticación ya consolidados.
- **c) Tras refactorizaciones arquitectónicas:** Al reestructurar clases, funciones o modelos de datos para mejorar su limpieza y eficiencia interna.
- **d) Al actualizar dependencias o infraestructura:** Cuando se migra a una versión superior de lenguaje (ej. Python 3.12 a 3.13), motor de base de datos o librerías del framework.
- **e) En cada integración y entrega continua (CI/CD):** Ejecutadas automáticamente ante cada solicitud de fusión (Pull Request) para autorizar despliegues a producción.

Estrategias y Herramientas Utilizadas en Pruebas de Regresión

A medida que una base de código evoluciona, el volumen total de casos de prueba puede tornarse masivo, haciendo inviable la ejecución manual periódica. Por ello, la ingeniería de software emplea tres estrategias formales de regresión [4], [5]:

- **1. Regresión Total (Retest-All):** Consiste en reejecutar la totalidad del catálogo de pruebas existente. Es la estrategia más exhaustiva y segura, pero demanda un alto consumo de cómputo y tiempo. Resulta factible principalmente cuando se cuenta con una suite de pruebas unitarias ligeras altamente automatizadas.
- **2. Selección de Pruebas de Regresión (Regression Test Selection - RTS):** Analiza las diferencias en el código fuente (commits/diffs) y determina qué módulos fueron potencialmente impactados. Se seleccionan únicamente los casos de prueba cuyas rutas de ejecución tocan los componentes modificados.
- **3. Priorización de Casos de Prueba (Test Case Prioritization - TCP):** Clasifica los casos de prueba asignándoles un orden de ejecución basado en criterios estratégicos: criticidad de negocio, probabilidad histórica de falla, cobertura de código alcanzada y tiempo de ejecución. Permite descubrir defectos en los primeros minutos de ejecución del pipeline.

Herramientas representativas: Frameworks de automatización como Pytest, JUnit, NUnit o TestNG (en capas unitarias y de integración); herramientas de navegación como Selenium WebDriver, Playwright y Cypress (para regresión de interfaces de usuario); y motores de integración continua como GitHub Actions, GitLab CI y Jenkins, encargados de orquestar y bloquear fusiones si algún test de regresión fracasa.

Beneficios y Desafíos de las Pruebas de Regresión

- **Beneficios Principales:** Brindan alta confianza y estabilidad para realizar entregas continuas; reducen drásticamente los costos de reparación al detectar regresiones en etapas tempranas; preservan la integridad histórica del sistema; y posibilitan el refactor audaz del código con una red de seguridad infalible.
- **Desafíos Críticos:** Crecimiento exponencial del tiempo de ejecución conforme el sistema se expande; presencia de pruebas intermitentes o inestables ("Flaky Tests") que fallan por factores ajenos al código (red, latencia de base de datos); y el costo de mantenimiento continuo requerido para sincronizar los scripts de prueba cuando los flujos de negocio son actualizados.

2. Ejemplos Prácticos Aplicados

Para consolidar los conceptos abordados en la investigación teórica, a continuación se describen tres escenarios técnicos detallados que ilustran la aplicación concreta de cada tipo de prueba en proyectos reales y representativos del sector industrial y de desarrollo [2], [4].

■ CASO PRÁCTICO 1: Sistema Bancario Sometido a Pruebas de Rendimiento y Estrés

Contexto del Sistema: Una institución bancaria opera una plataforma digital de transferencias interbancarias (tipo SPEI) y banca móvil que atiende a cientos de miles de cuentahabientes.

Planteamiento de la Prueba: Ante la proximidad del evento comercial del "Buen Fin" y el pago de quincenas, el equipo de aseguramiento de calidad (QA) diseña una prueba de rendimiento y estrés empleando la herramienta Apache JMeter.

Criterios de Prueba: Simular un escenario de carga máxima escalonada con 15,000 usuarios virtuales concurrentes ejecutando consultas de saldo, generación de transferencias y pagos de servicios durante un periodo continuo de 2 horas.

Hallazgo Crítico: A los 8,500 usuarios concurrentes, el tiempo de respuesta promedio de las transferencias se degradó de 250 milisegundos a 18.2 segundos. Al alcanzar los 11,000 usuarios, el servidor comenzó a arrojar errores HTTP 504 (Gateway Timeout) y se observó un bloqueo mutuo (deadlock) en la tabla transaccional de cuentas bancarias por contención en el pool de conexiones de la base de datos relacional.

Acción de Mitigación: Se rediseñó la arquitectura de persistencia mediante réplicas de lectura para las consultas de saldo, se implementó una cola asíncrona de mensajes con RabbitMQ para orquestar las transferencias en lotes controlados y se optimizó el pool de conexiones de 50 a 300 conexiones reutilizables. La nueva prueba de estrés certificó la estabilidad del sistema soportando 20,000 conexiones concurrentes con tiempos de respuesta inferiores a 1.2 segundos.

■ CASO PRÁCTICO 2: Detección, Registro y Ciclo de Vida de un Defecto Crítico en Jira

Contexto del Sistema: Sistema de Punto de Venta (POS) comercial para restaurantes y cafeterías.

Ficha Técnica de la Incidencia (Issue Key POS-184): Discrepancia en el cálculo del corte de caja diario al anular ventas con tarjeta bancaria. Módulo: Finanzas (/cortes/cierre). Severidad: Alta | Prioridad: P1 - Urgente.

Pasos para Reproducir: 1. Iniciar sesión como cajero en el POS. 2. Registrar venta de $450.00 MXN cobrada mediante Terminal Tarjeta. 3. Acceder al módulo de ventas y proceder a la anulación justificada. 4. Ir a Generar Corte de Caja y presionar "Calcular Totales Esperados".

Comportamiento Esperado vs Observado: Esperado: El total acumulado de ventas con tarjeta debe descontar la venta anulada, reflejando $0.00. Observado: El saldo acumulado sigue sumando los $450.00 de la venta cancelada, arrojando una diferencia artificial de caja y descuadrando el arqueo final.

Evidencia Anexa: Captura de pantalla del formulario de corte de caja y volcado del log con la consulta SQL subyacente.

Resolución y Trazabilidad: 1. El tester registra el ticket en estado "Nuevo". El líder asigna el ticket a un desarrollador backend ("Asignado"). 2. El desarrollador reproduce la falla y descubre que la consulta SUM(total) en modelsCorteCaja.py omitía el filtro WHERE estado != "CANCELADA" ("En Progreso"). 3. El programador corrige la consulta SQL, escribe una prueba unitaria específica en tests/test_modelos_corte_caja.py para validar el filtro de anulación y envía el Pull Request ("Resuelto"). 4. El tester despliega la versión a staging, repite los 4 pasos, confirma que el arqueo calcula $0.00 exactos y que las 290 pruebas unitarias pasan limpiamente ("Verificado"). 5. El ticket se actualiza formalmente a estado "Cerrado" con referencia al commit en la rama principal.

■ CASO PRÁCTICO 3: Pipeline CI/CD con Pruebas de Regresión Automatizadas ante Nueva Versión

Contexto del Sistema: Despliegue de la versión v2.4.0 de una plataforma de comercio web y punto de venta.

Escenario de Cambio: El equipo de desarrollo incorpora una nueva funcionalidad para otorgar promociones de "Hora Feliz" (Happy Hour) con descuento automático porcentual sobre productos de cafetería.

Riesgo de Regresión: La lógica de precios es transversal; una alteración incorrecta en la función compartida de cálculo de subtotal e impuestos podría quebrar la facturación regular de pedidos a domicilio o la generación de tickets de venta.

Ejecución del Pipeline en GitHub Actions: Al enviar el Pull Request a la rama main, se dispara automáticamente el workflow de Calidad con la suite completa de 290 pruebas unitarias y de integración en el runner de GitHub.

Alarma de Regresión Detectada: El test test_calculo_total_con_iva_y_descuento_general falló inesperadamente con código de error 1. La nueva función de "Hora Feliz" sobrescribía el valor del IVA general multiplicándolo por 0 en lugar de calcularlo sobre la base neta reducida.

Bloqueo Preventivo y Resolución: El pipeline bloqueó automáticamente el merge a producción, impidiendo emitir ventas con IVA incorrecto y evitando sanciones fiscales. El desarrollador ajustó la jerarquía de aplicación de impuestos en el modelo matemático, hizo push del parche, el pipeline corrió nuevamente y las 290 pruebas aprobaron con 100% de cobertura, permitiendo un despliegue seguro.

3. Análisis Reflexivo

Relevancia equivalente entre atributos funcionales y no funcionales: ¿Por qué las pruebas no funcionales son tan relevantes como las funcionales?

En el paradigma contemporáneo de la ingeniería de software, la división entre lo funcional y lo no funcional no debe interpretarse como una jerarquía de prioridades, sino como dos caras inseparables de la misma moneda. Un software que calcula una transacción con precisión funcional absoluta carece por completo de utilidad práctica si presenta un tiempo de respuesta de sesenta segundos, si sufre caídas ante el acceso concurrente de apenas veinte cajeros, o si posee brechas de seguridad que exponen las credenciales y los datos patrimoniales de los usuarios a actores maliciosos [1], [3]. La experiencia del usuario, la continuidad operativa del negocio, el cumplimiento de regulaciones de privacidad y la reputación de la organización descansan primordialmente en atributos no funcionales como el rendimiento, la resiliencia y la seguridad [4]. Por consiguiente, concebir un producto de software como "terminado" basándose exclusivamente en que sus botones y cálculos funcionan representa un grave error técnico que suele culminar en costosos fracasos comerciales.

El seguimiento de defectos como catalizador de la mejora continua: ¿Qué papel juega el seguimiento de defectos en la mejora continua del software?

El registro y seguimiento disciplinado de defectos trasciende por completo la simple noción de ser un "listado de pendientes por reparar". En el marco de la mejora continua (filosofías Kaizen, Agile y DevOps), el repositorio de incidencias constituye la principal fuente de inteligencia empírica del equipo de desarrollo [5], [6]. Analizar el historial de fallas permite identificar patrones sistemáticos de debilidad en la arquitectura, detectar módulos con alta densidad de defectos, evaluar la eficacia de los filtros de control de calidad y cuantificar métricas clave como el tiempo promedio de resolución y la tasa de reaparición de fallas. Más aún, cada defecto debidamente documentado impulsa el análisis de causa raíz (Root Cause Analysis - RCA), transformando el error humano en aprendizaje organizacional: se actualizan las pautas de codificación, se diseñan nuevas pruebas de regresión automatizadas para que el defecto nunca vuelva a manifestarse y se perfeccionan los procesos de diseño y estimación. De este modo, el seguimiento de defectos convierte los tropiezos del pasado en la garantía de robustez del futuro.

4. Referencias Bibliográficas (Formato IEEE)

[1] IEEE Standard for Software and System Test Documentation, IEEE Std 829-2008 (Revision of IEEE Std 829-1998), IEEE Computer Society, Piscataway, NJ, USA, pp. 1–128, Jul. 2008. doi: 10.1109/IEEESTD.2008.4578383.

[2] ISO/IEC/IEEE Systems and software engineering — Software testing — Part 1: Concepts and definitions, ISO/IEC/IEEE 29119-1:2022, International Organization for Standardization, Geneva, Switzerland, 2022.

[3] ISO/IEC Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, ISO/IEC 25010:2011, International Organization for Standardization, Geneva, Switzerland, 2011.

[4] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson Education, 2016, pp. 225–256.

[5] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020, pp. 480–522.

[6] G. J. Myers, C. Sandler, and T. Badgett, The Art of Software Testing, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, Inc., 2012, pp. 115–148.